Multi-Agent Essay Writing System - 5 Minute Demo Script

Demo Overview

- **Duration**: 5 minutes
- **Focus**: 6-stage multi-agent pipeline with human-in-the-loop checkpoints
- **Key Concepts**: LangGraph-inspired state machine, real-time collaboration, student profiles

---

SETUP BEFORE DEMO

1. Open two browser windows side-by-side (Teacher account + Student account)
2. Have VS Code ready with key files open in tabs
3. Create a new essay session with prompt: "*Reflect on a time when you questioned or challenged a belief or idea*"
4. Set target word count: 650 words

VOICE SCRIPT

[0:00 - 0:30] INTRODUCTION

> "Today I'm going to walk you through our multi-agent essay writing system built on a LangGraph-inspired architecture. This system uses six specialized AI agents that work sequentially, each with human-in-the-loop checkpoints where both teachers and students can approve, revise, or provide feedback in real-time."

[SHOW: Two browser windows side-by-side - Teacher and Student views]

> "What makes this unique is the real-time collaboration - you'll see both the teacher and student accounts stay perfectly synchronized as we progress through each stage."

[0:30 - 1:15] ARCHITECTURE OVERVIEW

[SHOW: VS Code - `constants.js`]

```
// Show lines 8-17
const STAGES = {
  NOT_STARTED: 'not_started',
  TOPIC_ANALYSIS: 'topic_analysis',
  RESEARCH: 'research',
  OUTLINE: 'outline',
  DRAFT: 'draft',
  EDITING: 'editing',
  POLISH: 'polish',
```

```
  COMPLETE: 'complete'
};
```

> "Our pipeline defines six processing stages. Each stage is a node in our state graph - similar to LangGraph's architecture where we have conditional edges and checkpoints."

[SHOW: VS Code - `pipeline.js` lines 344-380]

> "Here's our core orchestration. The `runStage` function retrieves the appropriate agent from our `STAGE_AGENTS` mapping, executes it with the current state, and emits events for real-time UI updates. After each stage, we create a checkpoint and await human approval before proceeding - this is the human-in-the-loop pattern."

```
// Highlight this code block:
async runStage(stage) {
  const agent = STAGE_AGENTS[stage];
  // ... state management
  this.emit('agentStarted', { sessionId, stage });
  const result = await agent(state);
  this.emit('agentOutput', { sessionId, stage, output: result });
  // Checkpoint for human approval
  this.emit('approvalRequested', { sessionId, stage });
}
```

[1:15 - 1:45] STUDENT PROFILE SELECTION

[SHOW: Browser - Student account on pre-start screen]

> "Before we start the pipeline, the student selects a profile that personalizes the entire essay experience. Let me select 'First-Generation College Student'."

[CLICK: First-Gen profile card, then 'Use This Profile']

> "Watch the teacher's screen - the profile syncs instantly via WebSocket."

[SHOW: Both screens showing the profile is saved]

> "This profile data flows through every agent, influencing topic suggestions, voice authenticity, and the final polish. The agents understand this student's unique perspective."

[1:45 - 2:30] STAGE 1: TOPIC ANALYSIS

[CLICK: 'Start Writing' button on Teacher screen]

> "The teacher initiates the pipeline. Our first agent is the Topic Analyzer."

[SHOW: VS Code - `topicAnalyzer.js` lines 17-45 (system prompt)]

> "Each agent has a specialized system prompt. The Topic Analyzer identifies multiple angles for approaching the essay prompt, considering the student's profile. It outputs structured JSON with potential angles, hooks, and risk assessments."

[SHOW: Browser - Topic Analysis output appearing on both screens]

> "Notice both screens update simultaneously. The agent has identified three potential angles. The student and teacher can now discuss which angle works best."

[SHOW: VS Code - `pipeline.js` approval flow]

> "Here's our approval checkpoint. The pipeline pauses and waits for `APPROVAL_STATUS.APPROVED` or `APPROVAL_STATUS.REVISION_REQUESTED`. If revision is requested, the agent re-runs with the feedback incorporated into its context."

[CLICK: Approve on Teacher screen]

[2:30 - 3:15] STAGES 2-3: RESEARCH & OUTLINE

> "With topic approved, we move to Research."

[SHOW: Browser - Research stage running]

> "The Research agent develops supporting content, identifies key moments and anecdotes, and creates a content strategy - all informed by the chosen angle and student profile."

[Brief pause as Research completes]

[SHOW: VS Code - `outlineArchitect.js` lines 20-50]

> "The Outline Architect is particularly interesting. It determines the narrative structure - whether to use 'in medias res', chronological, or a reflection frame. It creates a paragraph-by-paragraph blueprint with word count allocations."

[SHOW: Browser - Outline output with sections]

> "Look at the detailed structure: opening hook, specific sections with estimated word counts, transition guidance, and voice notes. This becomes the blueprint for our Draft agent."

[CLICK: Approve outline]

[3:15 - 4:00] STAGE 4: DRAFT WRITING

> "Now the Draft Writer takes over - this is where the magic happens."

[SHOW: VS Code - `draftWriter.js` system prompt lines 17-44]

> "Notice the voice requirements: 'Sound like an authentic 17-18 year old, NOT an adult, NOT AI.' We explicitly instruct against clichés like 'journey' or 'passion.' The agent uses 'show don't tell' techniques with sensory details."

[SHOW: Browser - Draft appearing in the collaborative editor]

> "The draft populates in our Liveblocks-powered collaborative editor. Both teacher and student can see cursor positions, make edits, and the changes sync in real-time."

[Type a few characters in Student window, show it appear in Teacher window]

> "See that? Real-time collaboration. Both participants can edit before approving."

[4:00 - 4:45] STAGES 5-6: EDITING & POLISH

[SHOW: Browser - Editing stage with suggestions panel]

> "The Editor Critic agent analyzes the draft and provides specific improvement suggestions - grammar fixes, clarity improvements, flow enhancements. Each suggestion shows the original text, the proposed change, and the reasoning."

[CLICK: Apply one suggestion]

> "Teachers and students can apply suggestions with one click. Watch - the change appears instantly in the editor on both screens."

[CLICK: Approve editing stage]

> "Finally, the Polish agent applies all remaining suggestions and performs a final quality pass, ensuring the essay scores 9.5+ on our internal quality metrics."

[SHOW: Browser - Final polished essay with score]

[4:45 - 5:00] CLOSING

[SHOW: VS Code - `essayPipelineService.js` event handlers]

> "To summarize: six specialized agents, each with human-in-the-loop checkpoints, real-time WebSocket synchronization between teacher and student, and personalized output based on student profiles. The LangGraph-inspired architecture gives us resumable pipelines, state persistence, and the flexibility to handle revisions at any stage."

> "This transforms essay writing from a solitary task into a collaborative, AI-assisted experience while keeping humans in control at every decision point."

KEY FILES TO SHOW (In Order)

Timestamp	File	Lines	Purpose
0:30	`services/essayAgents/constants.js`	8-30	Stage definitions, approval statuses

Timestamp	File	Lines	Purpose
0:45	`services/essayAgents/pipeline.js`	344-430	Core orchestration, `runStage()` function
1:45	`services/essayAgents/agents/topicAnalyzer.js`	17-60	Topic analysis system prompt
2:45	`services/essayAgents/agents/outlineArchitect.js`	20-70	Outline structure, narrative types
3:15	`services/essayAgents/agents/draftWriter.js`	17-44	Voice requirements, anti-AI patterns
4:45	`services/essayAgents/essayPipelineService.js`	36-80	Event forwarding, session management

KEY TECHNICAL POINTS TO MENTION

1. **State Machine Architecture**: "Like LangGraph, each stage is a node with conditional transitions based on approval status"
2. **Checkpoints**: "State is persisted to PostgreSQL (metadata) and MongoDB (large text content) at each checkpoint"
3. **Event-Driven Updates**: "Pipeline emits events (`agentStarted`, `agentOutput`, `approvalRequested`) that flow through Socket.IO to all connected clients"
4. **Token Optimization**: "Each agent has optimized token limits - Draft gets 4000 tokens, Topic Analysis only needs 2000"
5. **JSON Forcing**: "We use Claude's prefill technique with `{ role: 'assistant', content: '{' }` to guarantee structured JSON output"
6. **Profile Personalization**: "Student profile data (first-gen status, activities, strengths) flows through every agent's context"

DEMO FLOW CHECKLIST

- [ ] Two browsers open (Teacher + Student)
- [ ] VS Code with files tabbed
- [ ] New session created with Common App prompt
- [ ] Select student profile (First-Gen)
- [ ] Show profile sync between accounts
- [ ] Start pipeline from Teacher account
- [ ] Show Topic Analysis output on both screens
- [ ] Approve and progress through stages
- [ ] Show collaborative editing in Draft stage

- ☐ Apply editing suggestion
- ☐ Show final polished essay

BACKUP TALKING POINTS (If Questions Arise)

Q: Why not use LangGraph directly?

> "We built a LangGraph-inspired architecture optimized for our specific use case - collaborative human-in-the-loop workflows with WebSocket synchronization. The patterns are the same: nodes, edges, state, checkpoints."

Q: How do you handle errors?

> "Each agent returns gracefully on error with the error message in state. The pipeline can resume from the last successful checkpoint."

Q: Can students request revisions?

> "Students can provide feedback, but only teachers can officially approve or request revisions - maintaining the mentor/student dynamic."

Q: What model do you use?

> "Claude Sonnet 4 for the optimal balance of quality and speed. Each agent's system prompt is tuned for its specific task."